

UNIDADE SENAI 6.01 - SÃO CARLOS

GCD - Gerenciamento de Componentes Defeituosos

ESPECIFICAÇÃO DE REQUISITOS

ENTER DEVS

ÚLTIMA ATUALIZAÇÃO: 15/11/2025



GC&D

IDENTIFICAÇÃO DOS ENVOLVIDOS

PAPEL	NOME
PRODUCT OWNER	MIGUEL MANOEL <i>Responsável por construir o Backlog e acertar os detalhes de cada Sprint com os membros da equipe</i>
SCRUM MASTER	LUCAS PATRACÃO <i>Responsável por guiar o time na direção dos objetivos traçados</i>
STAKEHOLDER	SENAI 6.01 - Antonio Adolpho Lobbe <i>Instituição educacional solicitante do sistema</i>
FULL STACK	LUCAS PATRACÃO <i>Desenvolvimento completo da aplicação mobile</i>
DESIGNER	NYKOLAS ISLER <i>Criação do design system e interface do usuário</i>
HELPER DESIGNER	KAIO FARINA <i>Auxilia o designer nas tarefas de criação</i>

1. CONTEXTO

Diante da necessidade de modernizar o **gerenciamento de ferramentas e equipamentos** em laboratórios educacionais, especialmente nas **unidades SENAI**, é essencial que as instituições compreendam como utilizar ferramentas computacionais para oferecer **uma experiência diferenciada e eficiente no controle de componentes defeituosos**.

Atualmente, muitas instituições educacionais utilizam **métodos obsoletos de controle**, como **planilhas eletrônicas, registros em papel e comunicação via e-mail**, limitando o acesso às informações sobre o estado das ferramentas e dificultando a manutenção preventiva e corretiva.

De acordo com estudos sobre gestão de ativos em ambientes educacionais, **a falta de um sistema centralizado para gerenciamento de ferramentas resulta em:**

- **Perda de rastreabilidade** de defeitos
- **Dificuldade** na priorização de manutenções

- **Falta de histórico** de problemas por equipamento
- **Comunicação ineficiente** entre alunos, professores e administradores
- **Ausência** de métricas e análises sobre o estado do patrimônio

Para atender essa demanda, os alunos do curso de **Análise e Desenvolvimento de Sistemas** do **SENAI** foram selecionados para desenvolver um projeto com o objetivo de criar uma **aplicação mobile para o gerenciamento de componentes defeituosos** em laboratórios. A instituição solicitante necessita de um sistema mobile que permita o controle de ferramentas, salas e defeitos de forma prática, dinâmica e acessível.

A justificativa para esse projeto é que a instituição atualmente utiliza métodos obsoletos de controle, como planilhas e registros manuais, limitando o acesso às informações sobre ferramentas e defeitos. O foco da instituição é melhorar a gestão de seu patrimônio técnico, e a criação de uma plataforma mobile permitirá alcançar esse objetivo de maneira a:

- **Aumentar a rastreabilidade** de ferramentas e defeitos
 - **Melhorar a comunicação** entre alunos, professores e administradores
 - **Acelerar o processo** de reporte e validação de defeitos
 - **Facilitar a tomada de decisão** através de dashboards analíticos
 - **Reduzir o tempo de resposta** para manutenções
 - **Centralizar informações** em uma única plataforma
 - **Permitir acesso móvel** às informações de qualquer lugar
 - **Manter histórico completo** de defeitos e manutenções
 - **Facilitar a gestão de usuários** com diferentes níveis de acesso
-

2. OBJETIVOS

Este projeto tem como objetivo **desenvolver uma aplicação mobile para gestão de componentes defeituosos** em laboratórios educacionais, permitindo que alunos, professores e administradores gerenciem ferramentas, salas e defeitos de maneira eficiente e colaborativa.

2.1 Objetivo Geral

Desenvolver uma **aplicação mobile** multiplataforma (Android) utilizando React Native e Expo, que permita o **gerenciamento completo de ferramentas, salas e defeitos em laboratórios educacionais**, proporcionando uma experiência intuitiva e eficiente para todos os usuários.

2.2 Objetivos Específicos

- Implementar **sistema de autenticação com três níveis de acesso** (Aluno, Professor, Administrador)
- Desenvolver **módulo de gerenciamento de ferramentas com CRUD completo**
- Criar sistema de **reporte de defeitos com categorização e priorização**
- Implementar **dashboard analítico** com métricas e gráficos

- Desenvolver **sistema de gerenciamento de salas e designação de professores**
 - Criar **interface intuitiva** e responsiva seguindo design system consistente
 - Implementar **armazenamento local** persistente para dados
 - Desenvolver **sistema de notificações e feedback visual** (toast messages)
-

3. DESCRIÇÃO DO SISTEMA

O **GCD (Gerenciamento de Componentes Defeituosos)** é uma aplicação mobile desenvolvida em React Native com Expo, projetada para gerenciar ferramentas, salas e defeitos em laboratórios educacionais.

3.1 Arquitetura do Sistema

O sistema é composto por:

- **Frontend Mobile:** Aplicação React Native com Expo
- **Armazenamento Local:** AsyncStorage para persistência de dados
- **API Simulada:** Simulação de backend para operações CRUD
- **Context API:** Gerenciamento de estado global
- **React Navigation:** Sistema de navegação entre telas

3.2 Estrutura de Navegação

O sistema possui navegação diferenciada por tipo de usuário:

3.2.1 Navegação para Alunos

- **Home:** Painel principal com acesso rápido às funcionalidades
- **Dashboard:** Visualização de métricas gerais
- **Salas Disponíveis:** Lista de salas cadastradas
- **Ferramentas Disponíveis:** Lista de ferramentas para visualização
- **Acompanhar Defeitos:** Histórico de defeitos reportados pelo aluno
- **Perfil:** Informações pessoais e edição

3.2.2 Navegação para Professores

- **Home:** Painel principal com acesso rápido
- **Dashboard:** Métricas e análises
- **Gerenciar Salas:** CRUD completo de salas
- **Gerenciar Ferramentas:** CRUD completo de ferramentas
- **Ver Defeitos:** Lista completa de defeitos
- **Inbox de Defeitos:** Validação de defeitos pendentes
- **Perfil:** Informações pessoais

3.2.3 Navegação para Administradores

- **Home:** Painel principal
- **Dashboard:** Métricas completas do sistema
- **Gerenciar Salas:** CRUD de salas
- **Gerenciar Ferramentas:** CRUD de ferramentas
- **Monitorar Defeitos:** Visualização de todos os defeitos
- **Gerenciar Usuários:** Aprovação, edição e remoção de usuários
- **Perfil:** Informações pessoais

3.3 Fluxo de Autenticação

1. **Tela de Apresentação:** Carrossel com informações sobre o sistema
2. **Login/Cadastro:** Autenticação ou registro de novo usuário
3. **Status Pendente:** Tela exibida para usuários aguardando aprovação
4. **Navegação Principal:** Acesso ao sistema conforme tipo de usuário

3.4 Funcionalidades Principais

3.4.1 Gerenciamento de Ferramentas

- **Cadastro:** Nome, patrimônio, descrição, fabricante, modelo, número de série, data de aquisição, localização, status, fotos
- **Listagem:** Visualização de todas as ferramentas com filtros
- **Detalhes:** Informações completas, histórico de defeitos, galeria de fotos
- **Edição:** Atualização de informações da ferramenta
- **Exclusão:** Remoção de ferramentas do sistema
- **Reporte de Defeito:** Acesso rápido para reportar defeito em uma ferramenta

3.4.2 Sistema de Defeitos

- **Reporte:** Formulário completo com descrição, categoria, urgência, fotos, observações
- **Categorias:** Mecânico, Elétrico/Eletrônico, Funcional, Estético, Segurança, Outros
- **Status:** Pendente, Validado, Rejeitado, Em Reparo, Resolvido
- **Validação:** Professores validam ou rejeitam defeitos reportados
- **Histórico:** Visualização completa do histórico de defeitos por ferramenta
- **Inbox:** Professores visualizam defeitos pendentes de validação

3.4.3 Gerenciamento de Salas

- **Cadastro:** Nome, bloco, número, descrição, localização

- **Listagem:** Visualização de todas as salas
- **Detalhes:** Informações da sala e ferramentas associadas
- **Designação:** Administradores designam salas para professores
- **Associação:** Ferramentas são associadas a salas

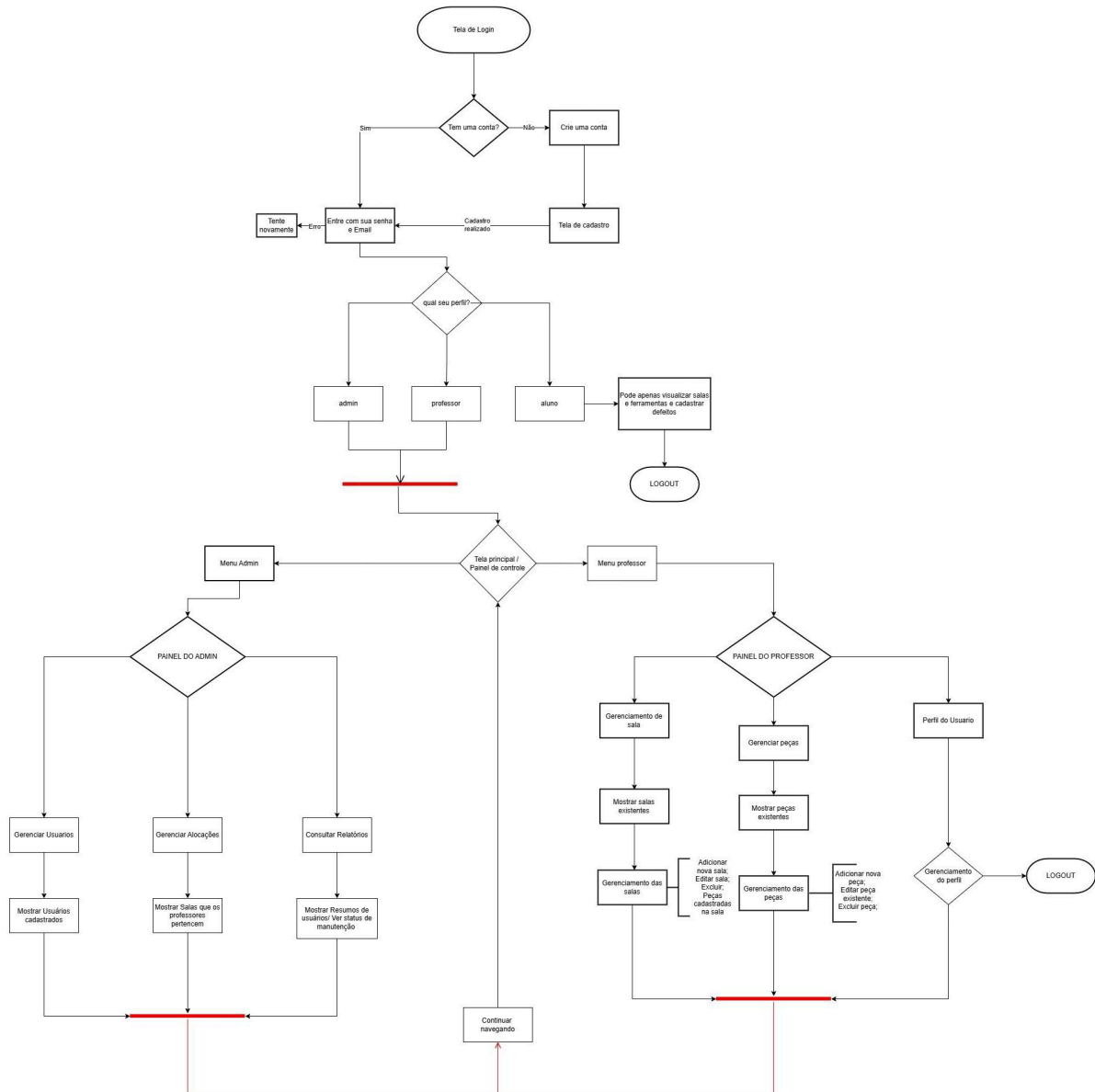
3.4.4 Gerenciamento de Usuários

- **Cadastro:** Nome, email, senha (todos começam como pendentes)
- **Aprovação:** Administradores aprovam e definem tipo (Aluno, Professor)
- **Edição:** Atualização de informações do usuário
- **Remoção:** Exclusão de usuários (exceto administrador padrão)
- **Filtros:** Filtro por tipo de usuário (Alunos, Professores, Administradores)

3.4.5 Dashboard Analítico

- **Métricas Gerais:** Total de ferramentas, salas, defeitos em aberto, usuários ativos
 - **Gráficos de Distribuição:** Ferramentas e defeitos por status
 - **Métricas Temporais:** Última hora, 24h, 7 dias, 30 dias
 - **Visualizações:** Gráficos de barras para melhor compreensão
-

4. FLUXOGRAMA



📁 projetos_fluxograma.jpg

5. LEVANTAMENTO DE REQUISITOS

De acordo com a descrição do sistema, levantou-se os seguintes requisitos:

5.1 GCD - Gerenciamento de Componentes Defeituosos - Regras de Negócio

5.1.1 Perfis dos Usuários

Aluno:

- Visualizar salas disponíveis
- Visualizar ferramentas disponíveis
- Reportar defeitos em ferramentas
- Acompanhar defeitos reportados por si mesmo
- Visualizar dashboard com métricas gerais
- Editar informações pessoais (nome e email)
- Não pode validar defeitos
- Não pode gerenciar ferramentas ou salas
- Não pode gerenciar outros usuários

Professor:

- **Todas as permissões de Aluno**
- Validar ou rejeitar defeitos reportados
- Gerenciar ferramentas (CRUD completo)
- Gerenciar salas (CRUD completo)
- Visualizar inbox de defeitos pendentes
- Designar salas para si mesmo
- Não pode gerenciar usuários
- Não pode aprovar novos usuários

Administrador:

- **Todas as permissões de Professor**
- Gerenciar usuários (aprovar, editar, remover)
- Designar salas para professores
- Acesso completo ao sistema
- Não pode remover o administrador padrão do sistema

5.1.2 Regras de Negócio - Autenticação

- Todos os novos usuários começam com status “pendente”
- Apenas administradores podem aprovar usuários
- Usuários pendentes não podem fazer login
- Usuários suspensos não podem fazer login
- Email deve ser único no sistema
- Senha mínima de 6 caracteres
- Recuperação de senha funcional localmente (sem envio de email)

5.1.3 Regras de Negócio - Ferramentas

- Nome e patrimônio são obrigatórios
- Sala deve ser selecionada no cadastro
- Status padrão: “disponível”
- Cada ferramenta pode ter até 5 fotos
- Ferramentas podem ser editadas por professores e administradores
- Ferramentas podem ser excluídas por professores e administradores
- Histórico de defeitos é mantido mesmo após exclusão da ferramenta

5.1.4 Regras de Negócio - Defeitos

- Descrição do problema é obrigatória
- Categoria do defeito é obrigatória
- Novo defeito sempre começa com status “pendente”
- Apenas professores e administradores podem validar/rejeitar defeitos
- Defeitos validados aparecem no histórico
- Defeitos rejeitados não aparecem no mural público
- Urgência imediata pode ser marcada pelo aluno
- Fotos são opcionais (até 5 fotos por defeito)
- Data/hora de identificação é registrada automaticamente

5.1.5 Regras de Negócio - Salas

- Nome da sala é obrigatório
- Bloco e número são opcionais
- Professores podem ser designados para múltiplas salas
- Administradores designam salas para professores
- Ferramentas são associadas a salas

5.1.6 Regras de Negócio - Usuários

- Email deve ser único
- Nome é obrigatório
- Tipo de usuário é definido pelo administrador na aprovação
- Status pode ser: pendente, aprovado, suspenso
- Administrador padrão não pode ser removido
- Usuários podem editar apenas nome e email (não tipo ou status)

5.2 Requisitos Funcionais

RF01 - Autenticação e Autorização

- **RF01.1:** Sistema deve permitir cadastro de novos usuários
- **RF01.2:** Sistema deve validar credenciais no login
- **RF01.3:** Sistema deve permitir recuperação de senha

- **RF01.4:** Sistema deve controlar acesso por tipo de usuário
- **RF01.5:** Sistema deve impedir login de usuários pendentes ou suspensos

RF02 - Gerenciamento de Ferramentas

- **RF02.1:** Sistema deve permitir cadastro de ferramentas
- **RF02.2:** Sistema deve listar todas as ferramentas
- **RF02.3:** Sistema deve exibir detalhes completos da ferramenta
- **RF02.4:** Sistema deve permitir edição de ferramentas
- **RF02.5:** Sistema deve permitir exclusão de ferramentas
- **RF02.6:** Sistema deve validar campos obrigatórios
- **RF02.7:** Sistema deve associar ferramentas a salas
- **RF02.8:** Sistema deve manter histórico de defeitos por ferramenta

RF03 - Sistema de Defeitos

- **RF03.1:** Sistema deve permitir reporte de defeitos
- **RF03.2:** Sistema deve categorizar defeitos
- **RF03.3:** Sistema deve permitir anexo de fotos
- **RF03.4:** Sistema deve permitir validação de defeitos
- **RF03.5:** Sistema deve permitir rejeição de defeitos
- **RF03.6:** Sistema deve manter histórico de defeitos
- **RF03.7:** Sistema deve exibir inbox de defeitos pendentes
- **RF03.8:** Sistema deve permitir marcação de urgência

RF04 - Gerenciamento de Salas

- **RF04.1:** Sistema deve permitir cadastro de salas
- **RF04.2:** Sistema deve listar todas as salas
- **RF04.3:** Sistema deve exibir detalhes da sala
- **RF04.4:** Sistema deve permitir edição de salas
- **RF04.5:** Sistema deve permitir exclusão de salas
- **RF04.6:** Sistema deve permitir designação de professores a salas
- **RF04.7:** Sistema deve exibir ferramentas associadas à sala

RF05 - Gerenciamento de Usuários

- **RF05.1:** Sistema deve listar todos os usuários
- **RF05.2:** Sistema deve permitir aprovação de usuários
- **RF05.3:** Sistema deve permitir edição de usuários
- **RF05.4:** Sistema deve permitir remoção de usuários
- **RF05.5:** Sistema deve filtrar usuários por tipo
- **RF05.6:** Sistema deve exibir detalhes do usuário

RF06 - Dashboard e Relatórios

- **RF06.1:** Sistema deve exibir métricas gerais
- **RF06.2:** Sistema deve exibir gráficos de distribuição
- **RF06.3:** Sistema deve permitir filtro por período temporal
- **RF06.4:** Sistema deve calcular estatísticas em tempo real

RF07 - Perfil do Usuário

- **RF07.1:** Sistema deve exibir informações do usuário logado
- **RF07.2:** Sistema deve permitir edição de nome e email
- **RF07.3:** Sistema deve permitir logout

5.3 Requisitos Não Funcionais

RNF01 - Performance

- Sistema deve carregar telas em menos de 2 segundos
- Operações CRUD devem ser executadas em menos de 1 segundo
- Scroll deve ser fluido sem travamentos

RNF02 - Usabilidade

- Interface deve ser intuitiva e fácil de usar
- Navegação deve ser clara e consistente
- Feedback visual deve ser fornecido para todas as ações
- Mensagens de erro devem ser claras e objetivas

RNF03 - Compatibilidade

- Sistema deve funcionar em Android 6.0 ou superior
- Sistema deve funcionar em iOS 12.0 ou superior
- Sistema deve ser responsivo a diferentes tamanhos de tela

RNF04 - Segurança

- Senhas não devem ser exibidas em texto plano
- Sessões devem ser gerenciadas adequadamente
- Validação de dados deve ser realizada no frontend

RNF05 - Manutenibilidade

- Código deve seguir padrões de boas práticas

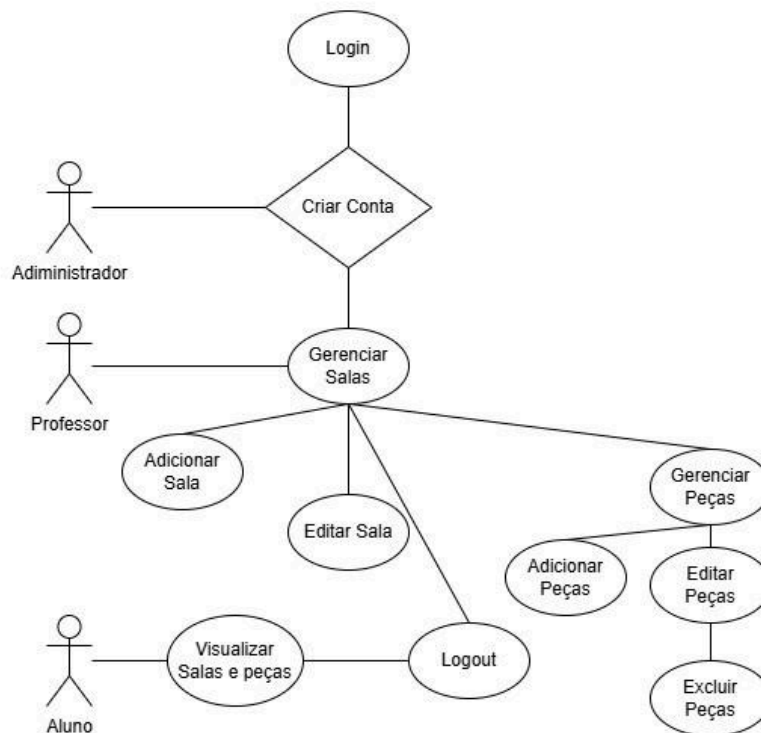
- Componentes devem ser reutilizáveis
- Documentação deve estar atualizada

6. PRODUCT BACKLOG

+ Gerenciador de Tarefas/Tempo (Projeto GC&D)

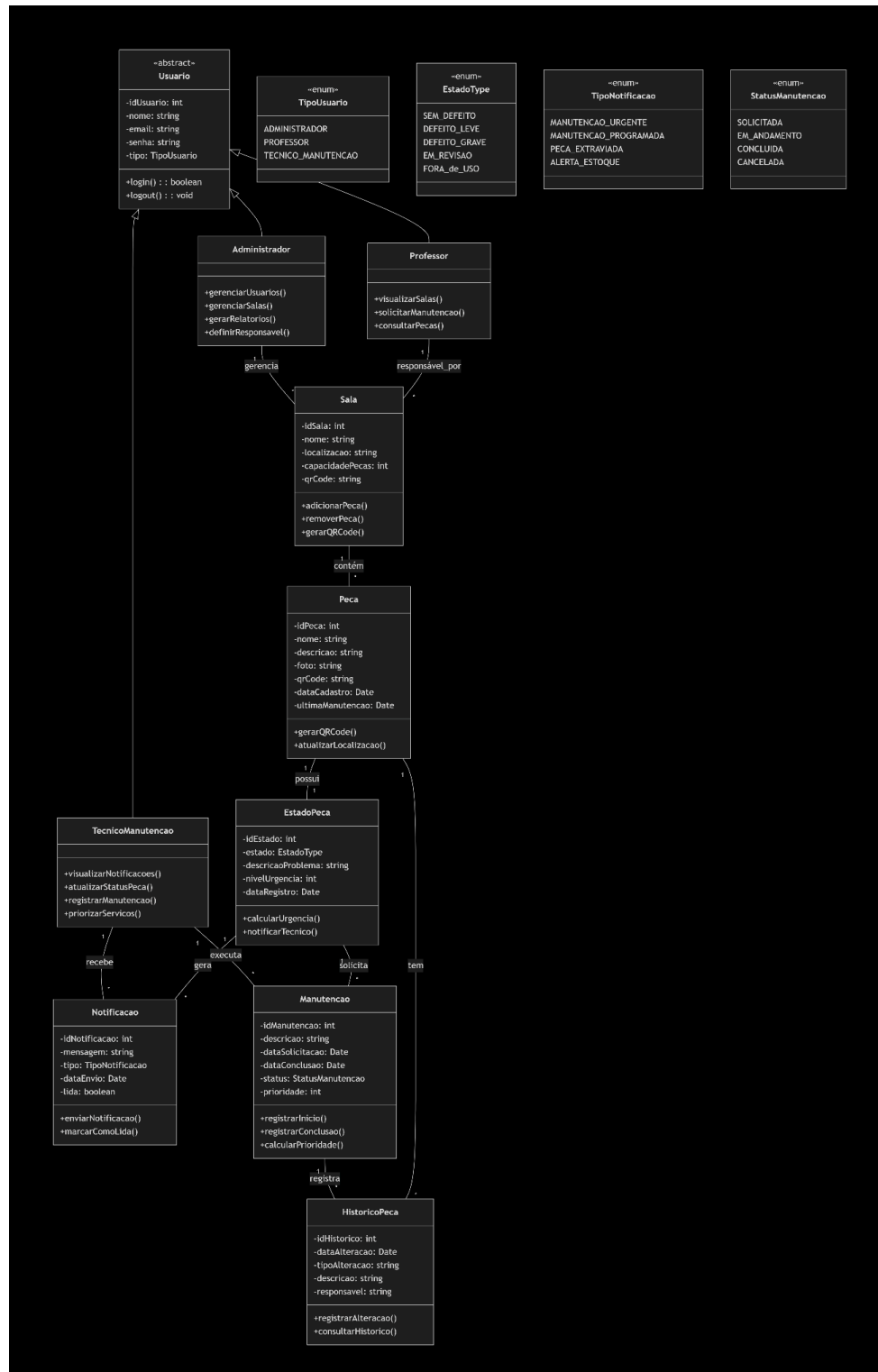
7. DIAGRAMAS UML

7.1 Diagrama de Casos de Uso



📁 projetos_diagramaCasoDeUso.jpg

7.2 Diagrama de Classes



📄 projetos_diagramaClasse.png

8. PROTOTIPAÇÃO DAS INTERFACES

8.1 Tela de Apresentação

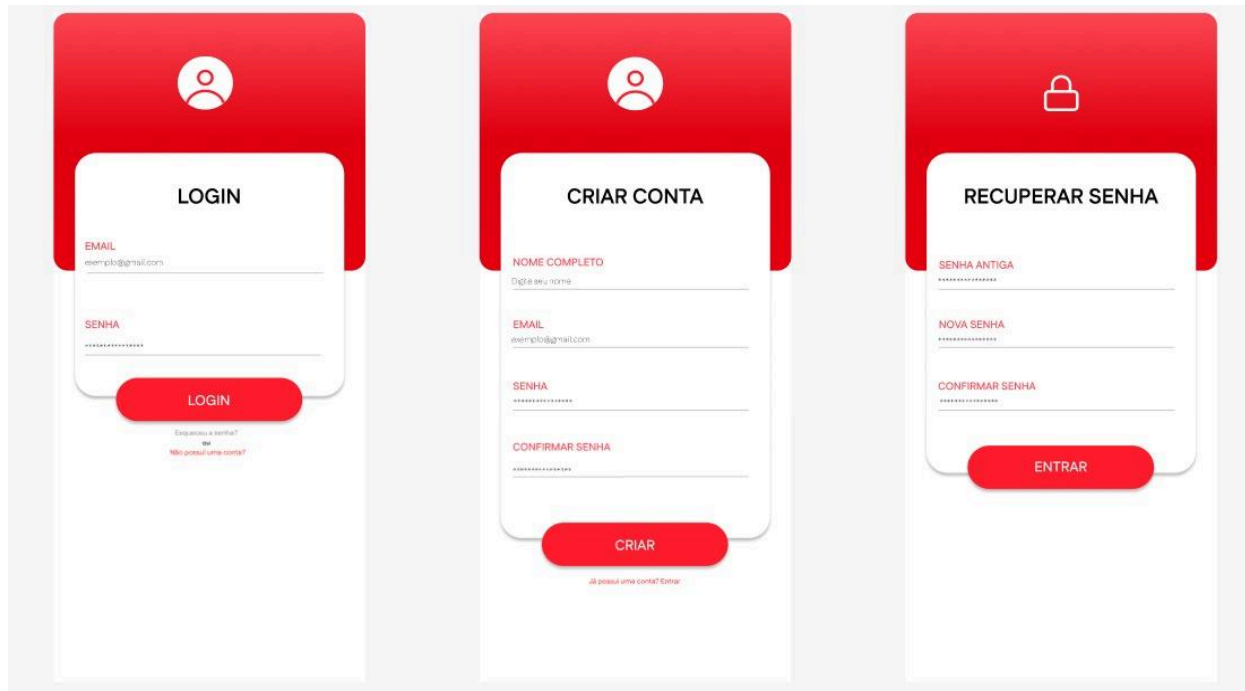
- Carrossel com 3 slides explicativos
- Botão “Pular” no canto superior direito
- Botão “Continuar” ou “Começar agora”
- Link para cadastro na parte inferior
- Paginação visual (dots) indicando slide atual

8.2 Tela de Login

- Campo de email
- Campo de senha
- Botão “Entrar”
- Link “Esqueci minha senha”
- Link “Não tem conta? Cadastre-se”

8.3 Tela de Cadastro

- Campo nome completo
- Campo email
- Campo senha
- Campo confirmar senha
- Botão “Cadastrar”
- Link para voltar ao login



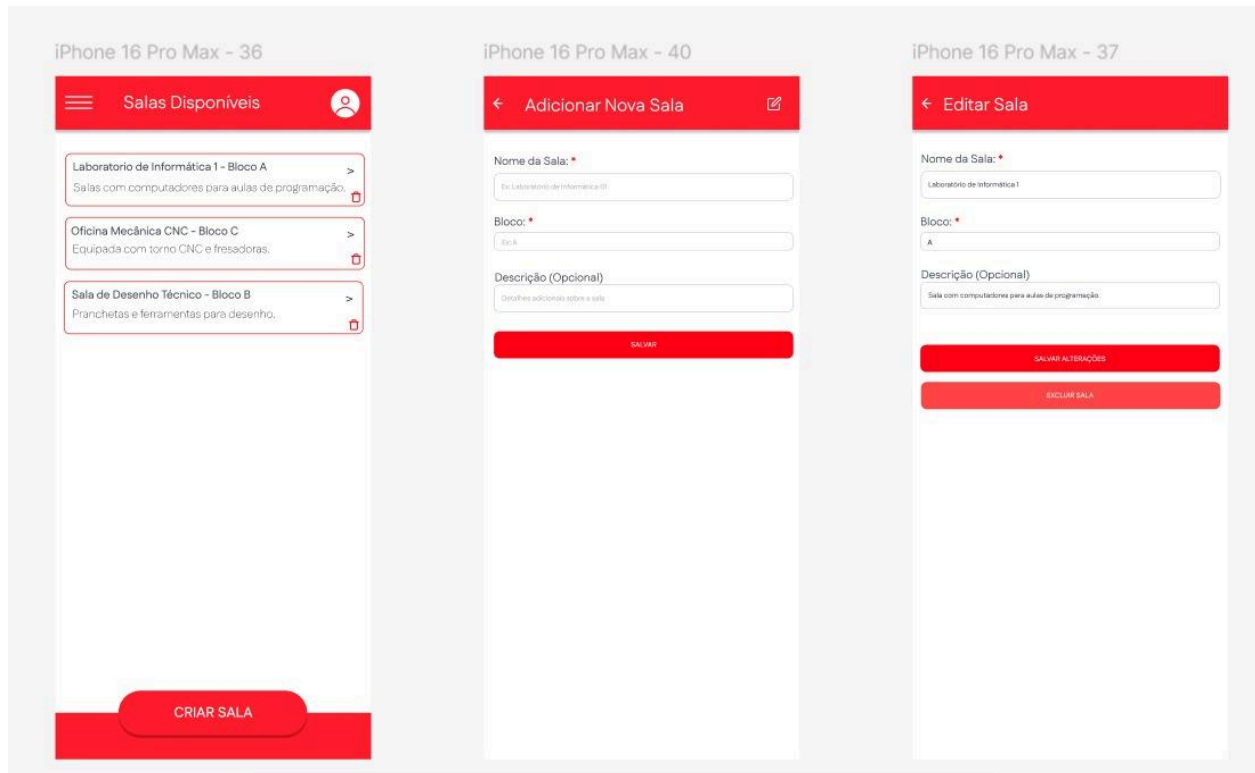
8.4 Painel Principal (Home)

- Header com gradiente e saudação personalizada
- Grid de cards com ações rápidas:
 - Dashboard
 - Salas
 - Ferramentas
 - Defeitos
 - (Varia conforme tipo de usuário)



8.5 Dashboard

- Header com métricas principais (cards)
- Seletor de intervalo temporal (1h, 24h, 7d, 30d)
- Cards de informação com gráficos de barras
- Distribuição de ferramentas por status
- Distribuição de defeitos por status

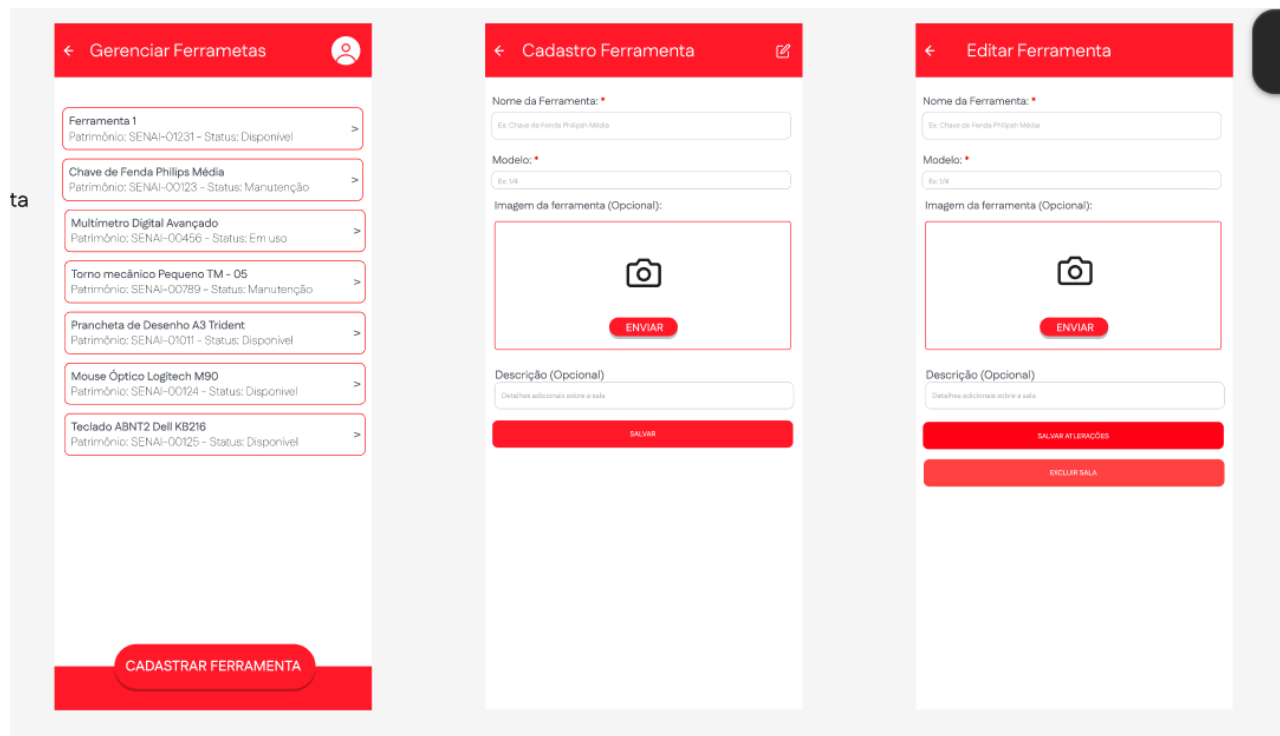


8.6 Lista de Ferramentas

- Header com título e contador
- Cards de ferramentas com:
 - Nome
 - Patrimônio
 - Status (tag colorida)
 - Sala associada
- Botão flutuante para adicionar nova ferramenta
- Pull-to-refresh

8.7 Detalhes da Ferramenta

- Header com gradiente contendo:
 - Nome da ferramenta
 - Patrimônio
 - Status
 - Métricas (localização, último uso, defeitos)
- Galeria de fotos
- Informações gerais (fabricante, modelo, etc.)
- Histórico de defeitos
- Botão flutuante para reportar defeito



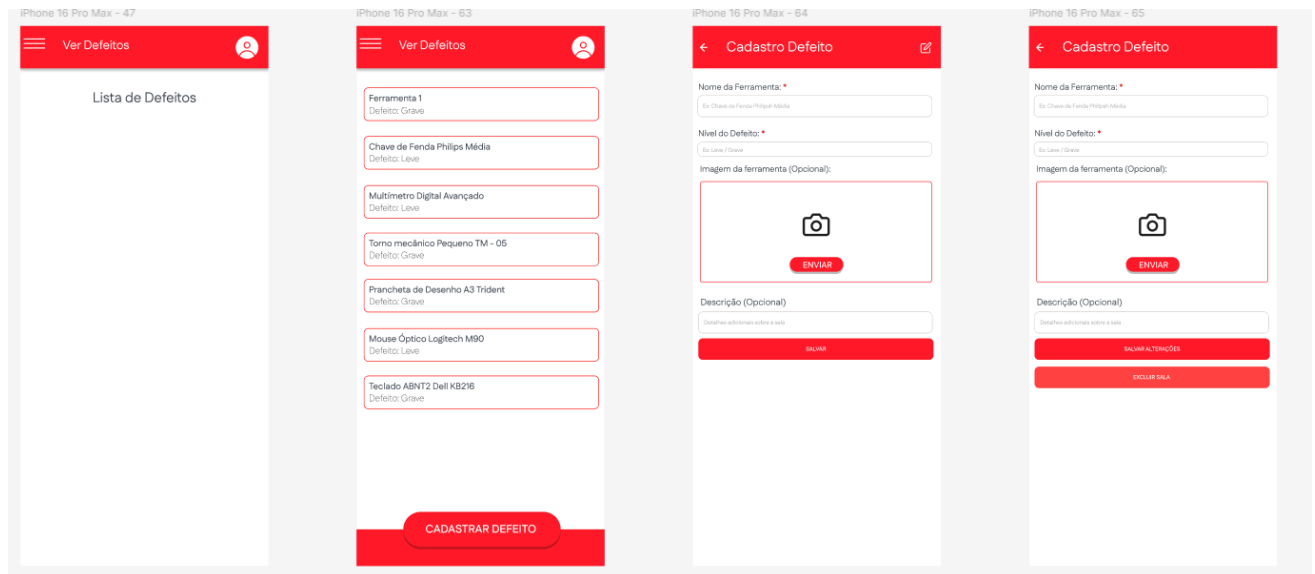
8.8 Formulário de Defeito

- Seção: Ferramenta (picker)
- Seção: Categoria (picker com descrição)
- Seção: Descrição (textarea)
- Seção: Observações (textarea opcional)
- Seção: Anexos (botão para adicionar fotos)
- Seção: Urgência (switch)
- Botão “Registrar Defeito”

8.9 Inbox de Defeitos (Professores)

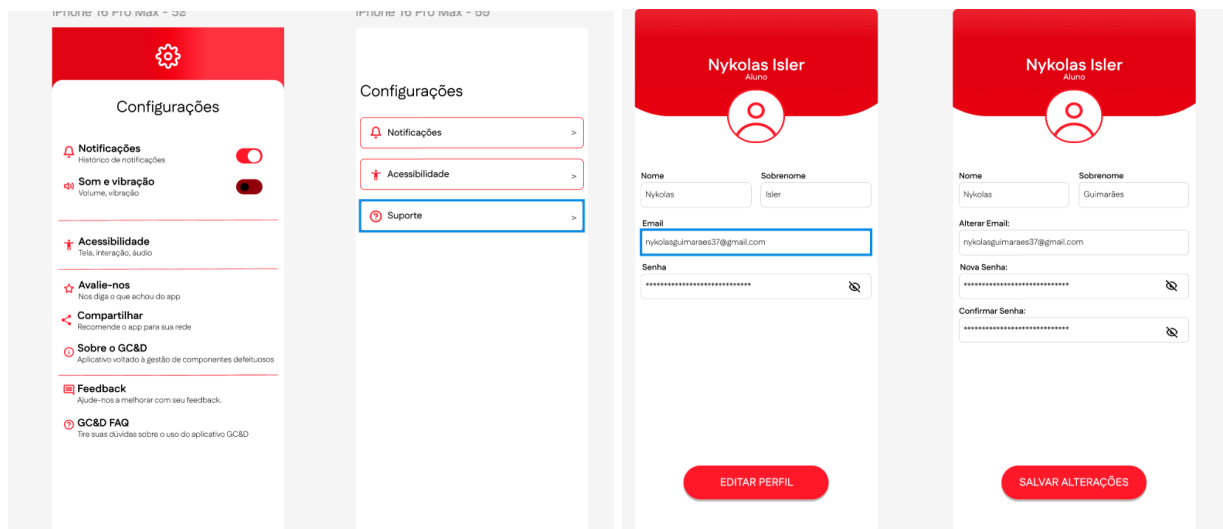
- Header com métricas (pendentes, validados, urgentes)
- Card de filtros:
 - Filtro por status (todos, pendentes, validados)
 - Filtro por categoria
 - Filtro por prioridade
 - Toggle “Somente meus”
- Lista de defeitos com:
 - Nome da ferramenta
 - Descrição
 - Status e tags

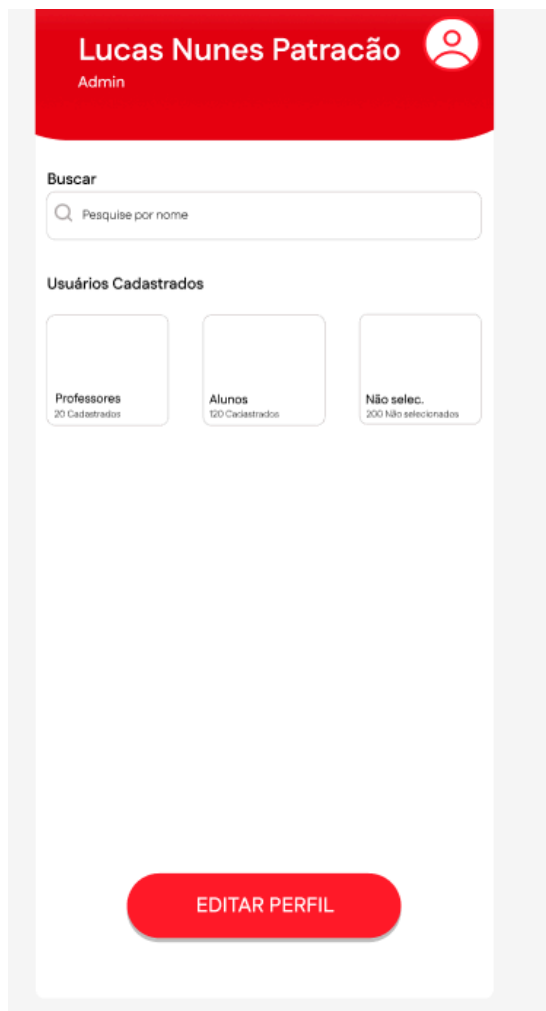
- Botões de ação (Validar/Rejeitar)



8.10 Gerenciar Usuários (Administradores)

- Header com estatísticas (total, pendentes, aprovados)
- Card de filtros por tipo de usuário
- Lista de usuários com:
 - Nome e email
 - Tags de status e tipo
 - Botões de ação (Aprovar, Designar Salas)
- Loading overlay ao navegar para detalhes



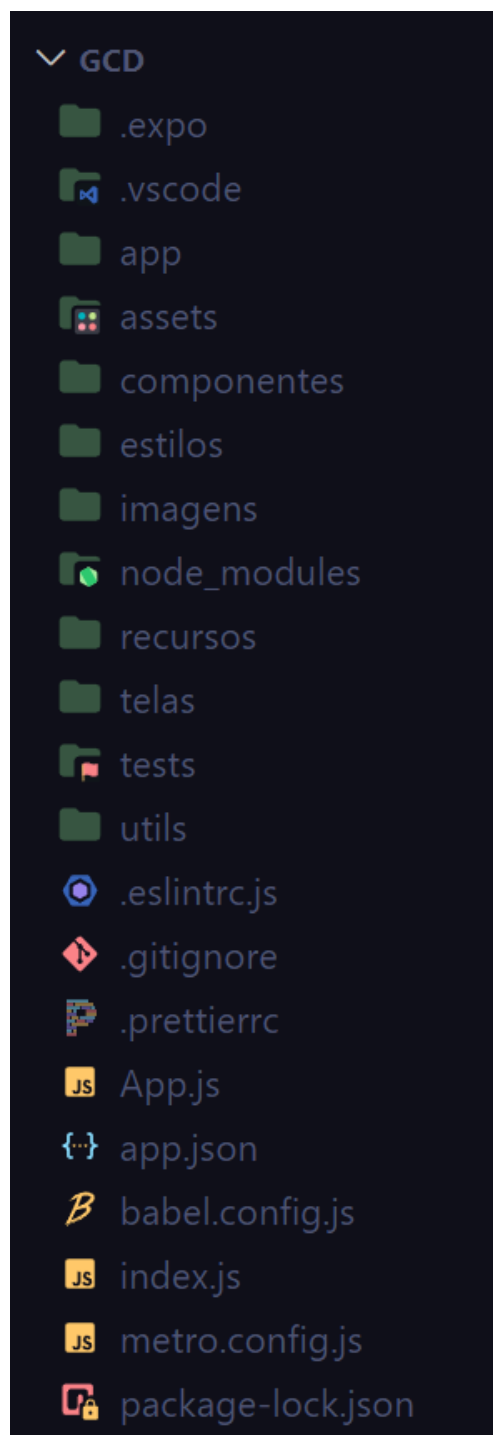


9. SOFTWARE DESENVOLVIDO

9.1 Tecnologias Utilizadas

- **React Native 0.73.6:** Framework para desenvolvimento mobile
- **Expo SDK 50:** Plataforma de desenvolvimento
- **React Navigation 6.x:** Sistema de navegação
- **Styled Components 6.1.19:** Estilização CSS-in-JS
- **AsyncStorage 1.21.0:** Armazenamento local persistente
- **Expo Image Picker 14.7.1:** Seleção de imagens
- **React Native Picker 2.6.1:** Componentes de seleção
- **Expo Linear Gradient 12.7.2:** Gradientes visuais
- **Material Community Icons:** Biblioteca de ícones

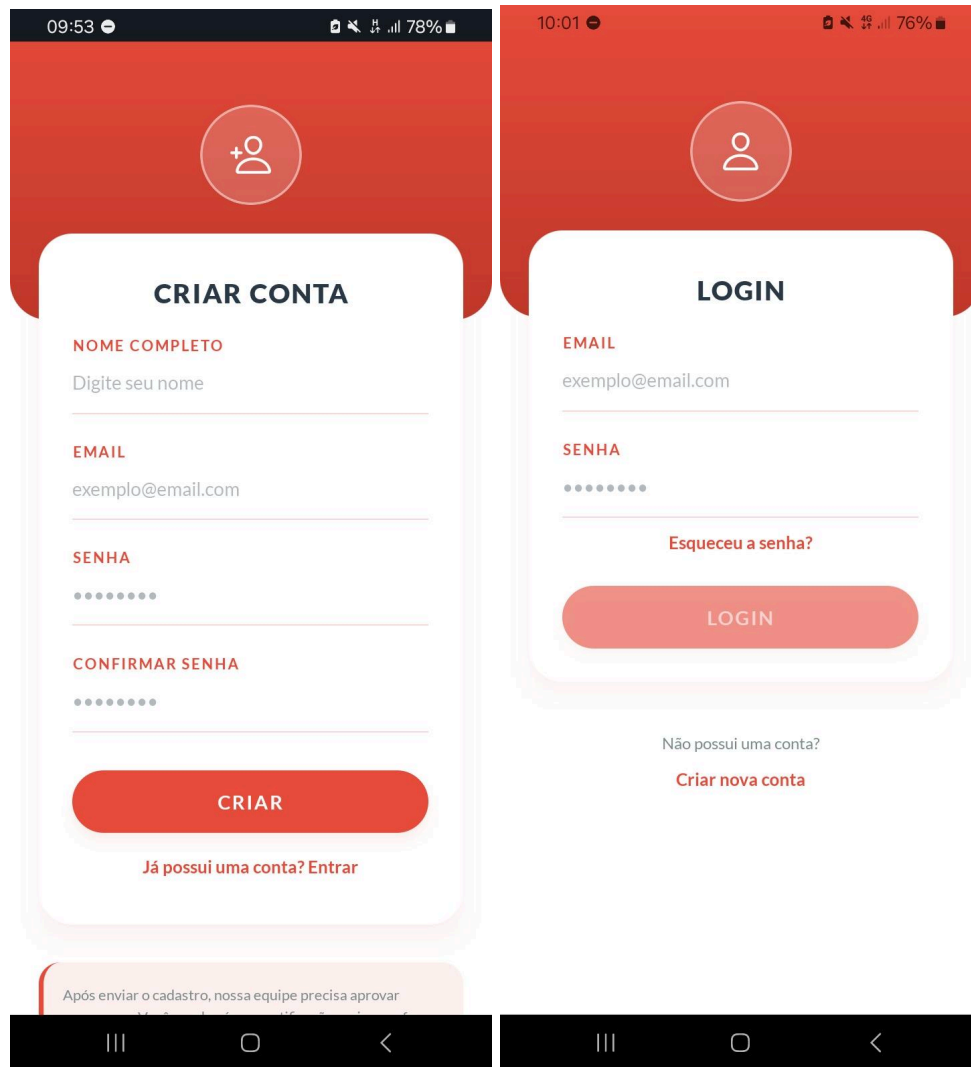
9.2 Estrutura de Pastas



9.3 Telas Implementadas

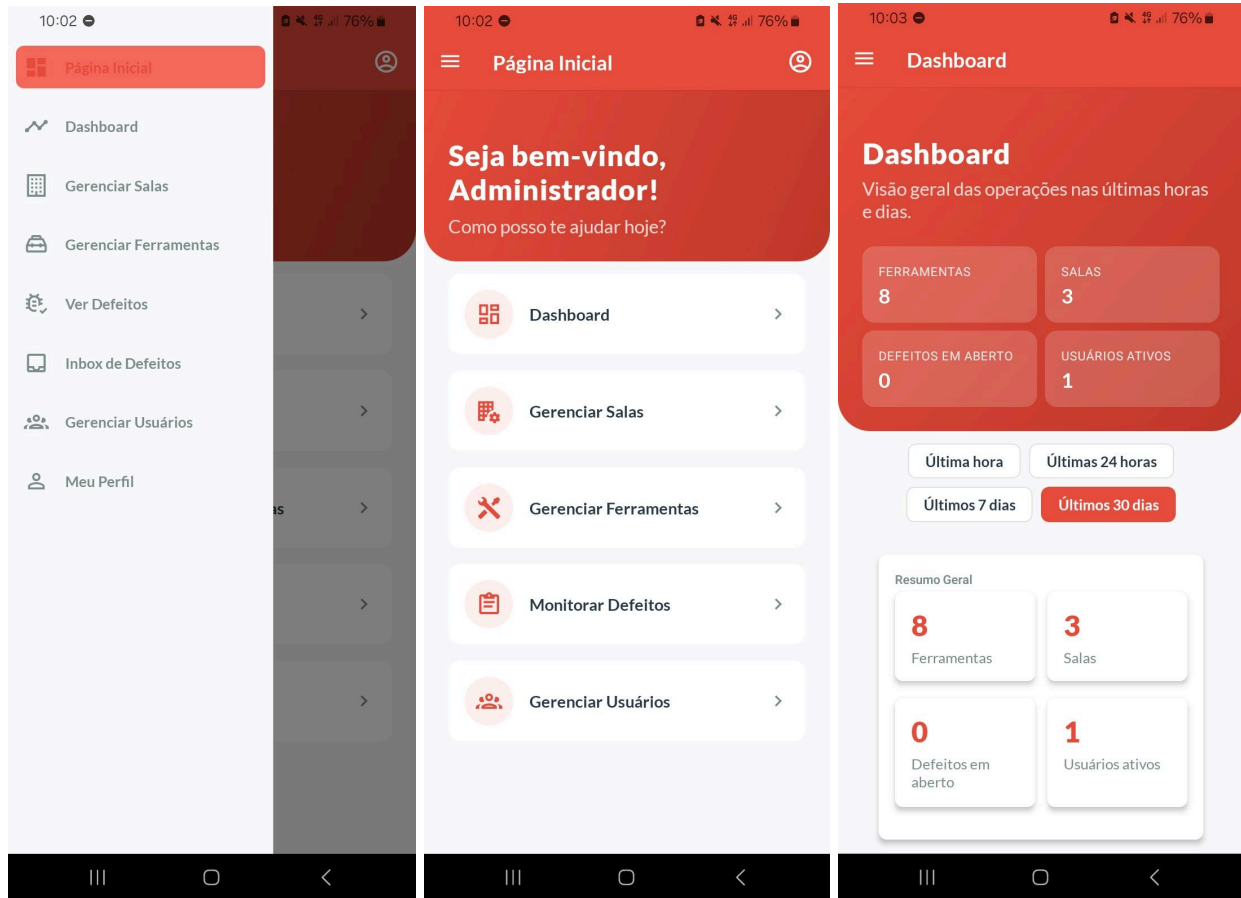
Autenticação:

1. **ApresentacaoTela:** Carrossel de introdução
2. **LoginTela:** Autenticação de usuários
3. **CadastroTela:** Registro de novos usuários
4. **RecuperarSenhaTela:** Redefinição de senha
5. **StatusPendenteTela:** Tela para usuários aguardando aprovação



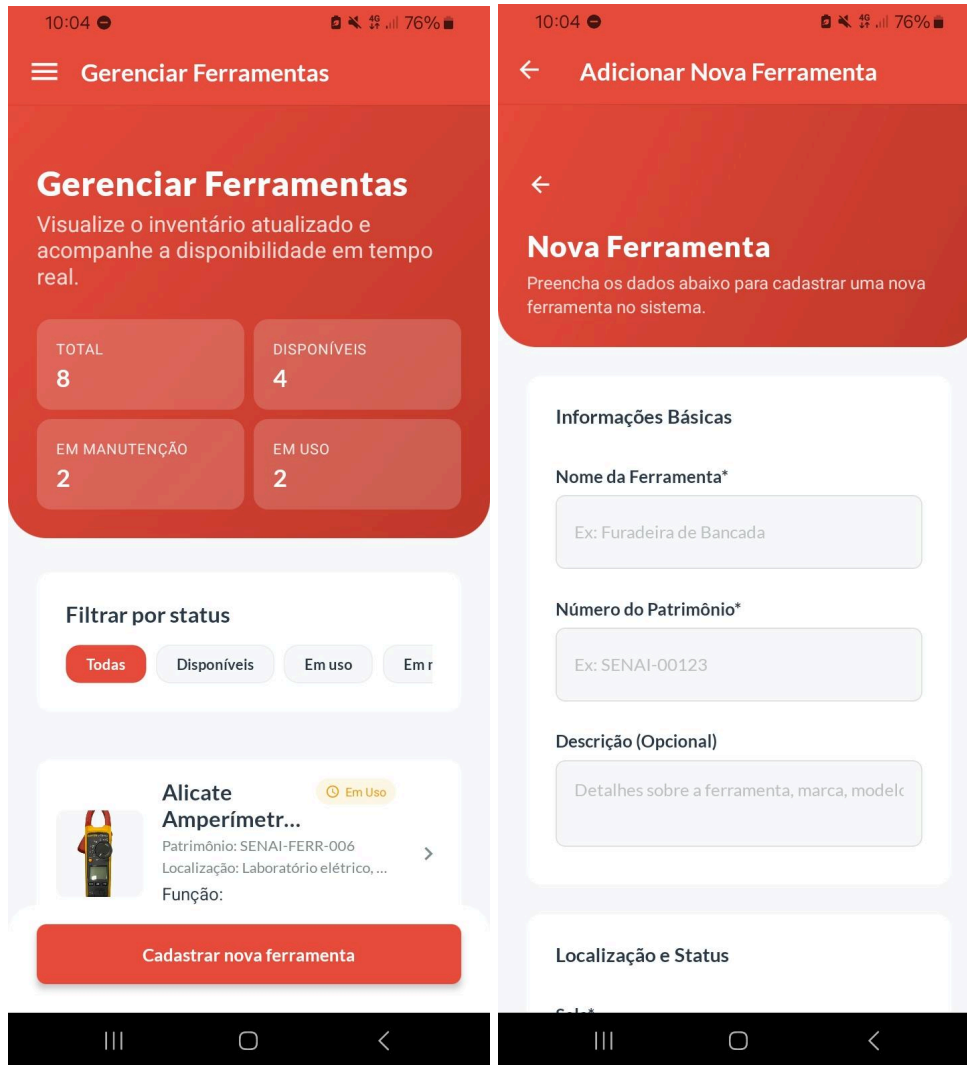
Home:

1. **PainelPrincipalTela:** Acesso rápido às funcionalidades
2. **DashboardTela:** Métricas e gráficos analíticos



Ferramentas:

1. **ListaFerramentasTela**: Listagem de ferramentas
2. **DetalhesFerramentaTela**: Detalhes completos
3. **FormularioFerramentaTela**: Cadastro/edição
4. **ListaFerramentasDisponiveisTela**: Visualização para alunos



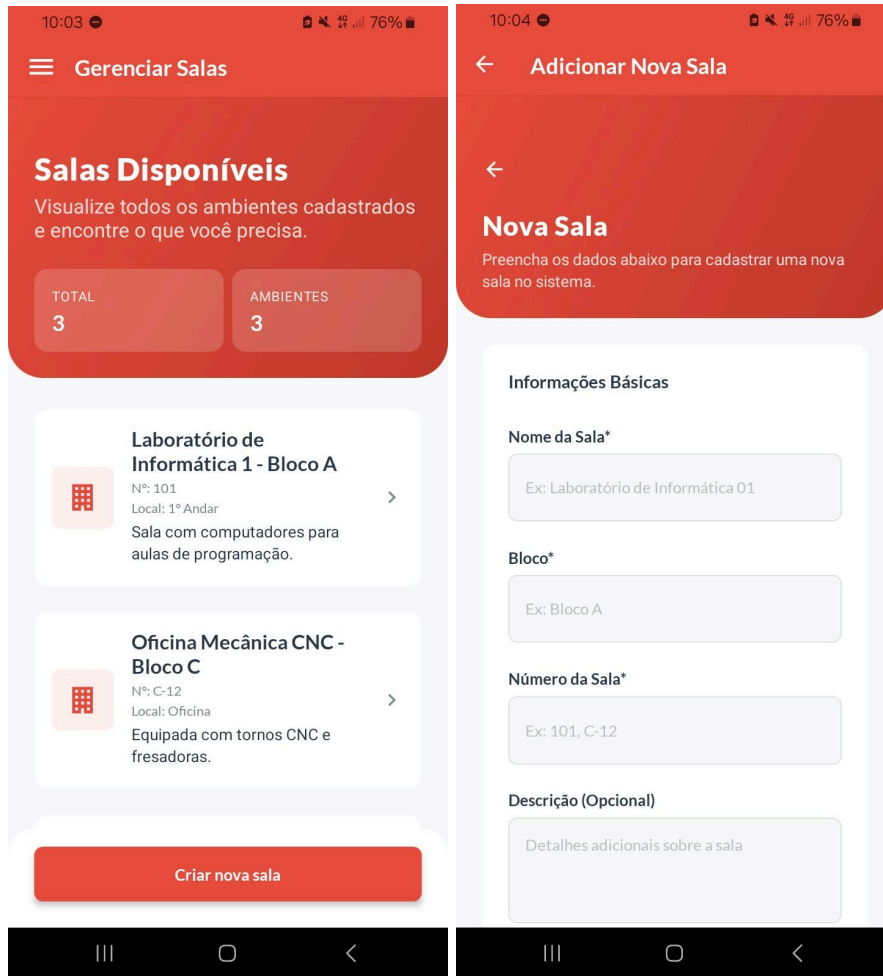
Defeitos:

1. **FormularioDefeitoMelhoradoTela:** Reporte completo
2. **ListaDefeitosTela:** Histórico de defeitos
3. **InboxProfessorTela:** Validação de defeitos
4. **DetalhesDefeitoInboxTela:** Detalhes para validação
5. **ReportarDefeitoTela:** Tela alternativa de reporte



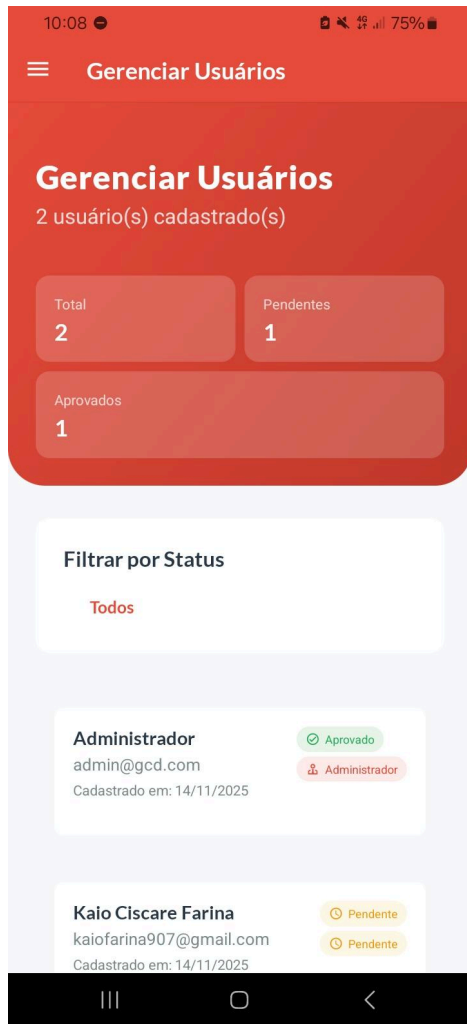
Salas:

1. **ListaSalasTela:** Listagem de salas
2. **DetalhesSalaTela:** Detalhes da sala
3. **FormularioSalaTela:** Cadastro/edição
4. **VisualizarSalaAlunoTela:** Visualização para alunos



Administração:

1. **ListaUsuariosTela:** Gerenciamento de usuários
2. **DetalhesUsuarioTela:** Detalhes e edição
3. **DesignarSalasTela:** Designação de salas para professores



Perfil:

1. **PerfilTela:** Informações pessoais e edição

9.4 Componentes Reutilizáveis

- **BotaoPrimario:** Botão principal padronizado
- **TagStatus:** Tags de status coloridas
- **CardDefeito:** Card para exibição de defeitos
- **ModalDetalhesDefeito:** Modal detalhado de defeitos
- **EstadoFeedback:** Componente para estados (carregando, erro, vazio)
- **ContainerComScroll:** Container com scroll automático
- **EntradaTextoEstilizada:** Inputs padronizados
- **RotuloEstilizado:** Labels padronizados

9.5 Funcionalidades Implementadas

- Sistema de autenticação completo
 - Gerenciamento de usuários com três níveis
 - CRUD completo de ferramentas
 - CRUD completo de salas
 - Sistema de reporte e validação de defeitos
 - Dashboard analítico com gráficos
 - Sistema de filtros e buscas
 - Armazenamento local persistente
 - Design system consistente
 - Navegação por tipo de usuário
 - Recuperação de senha local
 - Galeria de fotos
 - Histórico completo de defeitos
 - Métricas temporais
 - Notificações toast
-

10. RECURSOS

10.1 Pessoal

- **Desenvolvedores Full Stack:** Para desenvolvimento completo da aplicação mobile
- **Designer UI/UX:** Para criação do design system e interface
- **Product Owner:** Para definição de requisitos e prioridades
- **Scrum Master:** Para gestão ágil do projeto
- **Equipe de Testes:** Para validação de funcionalidades (opcional)

10.2 Tecnologia

- **Ambiente de Desenvolvimento Integrado (IDE):**
 - Visual Studio Code ou similar
 - Extensões: React Native Tools, ESLint, Prettier
- **Sistemas de Versionamento:**
 - Git para controle de versão
 - GitHub para hospedagem do repositório
- **Ferramentas de Desenvolvimento:**
 - Node.js e npm/yarn
 - Expo CLI
 - React Native Debugger
 - Flipper (opcional)
- **Serviços em Nuvem (Futuro):**
 - Backend as a Service (Firebase, Supabase)

- Armazenamento de imagens (Cloudinary, AWS S3)
- Autenticação (Auth0, Firebase Auth)
- **Ferramentas de Teste:**
 - Jest para testes unitários
 - React Native Testing Library
 - Detox para testes E2E (opcional)

10.3 Equipamentos

- **Computadores/Notebooks:**
 - Mínimo: 8GB RAM, processador multi-core
 - Recomendado: 16GB RAM, SSD
- **Dispositivos Móveis para Testes:**
 - Android: Dispositivo físico ou emulador
 - iOS: Dispositivo físico ou simulador (apenas macOS)
- **Acessórios:**
 - Cabos USB para conexão de dispositivos
 - Contas de desenvolvedor (Google Play, App Store - futuro)

10.4 Comunicação

- **Ferramentas de Colaboração:**
 - GitHub para versionamento e issues
 - Plataforma de comunicação (Discord, Slack, Teams)
 - Documentação compartilhada (GitHub Wiki, Notion)
-

11. CUSTOS

11.1 Pessoal

Desenvolvedores:

- Equipe de desenvolvimento trabalhando no projeto
- Custo: \$0 (projeto acadêmico)

Designer:

- Criação de design system e interface
- Custo: \$0 (projeto acadêmico)

Gerente de Projeto:

- Coordenação e supervisão
- Custo: \$0 (projeto acadêmico)

11.2 Tecnologia

Ambiente de Desenvolvimento Integrado (IDE):

- Visual Studio Code (gratuito)
- Extensões (gratuitas)
- **Custo total: \$0**

Sistemas de Versionamento:

- GitHub (plano gratuito)
- Git (open source)
- **Custo total: \$0**

Servidores e Infraestrutura em Nuvem:

- **Atualmente:**
 - **Armazenamento local (AsyncStorage)** - \$0
- **Futuro (produção):**
 - **Backend:** \$20-50/mês (Firebase, Supabase)
 - **Armazenamento de imagens:** \$10-30/mês

Custo estimado para produção: \$30-80/mês

Ferramentas de Teste:

- Jest (open source)
- React Native Testing Library (open source)
- **Custo total: \$0**

Ferramentas de Build:

- Expo EAS Build (plano gratuito disponível)
- Builds ilimitados no plano pago: \$29/mês
- **Custo atual: \$0 | Produção: \$29/mês (opcional)**

11.3 Equipamentos

Computadores e Notebooks:

- Utilização de equipamentos existentes do SENAI
- **Custo: \$0**

Dispositivos Móveis para Testes:

- Utilização de dispositivos pessoais ou do laboratório

- Emuladores/simuladores (gratuitos)
- **Custo: \$0**

Contas de Desenvolvedor:

- Google Play Developer: \$25 (taxa única)
- Apple Developer Program: \$99/ano
- **Custo atual: \$0 | Produção: \$25-99/ano**

11.4 Resumo de Custos

CATEGORIA	CUSTO ATUAL	CUSTO PRODUÇÃO (MENSAL)
PESSOAL	\$0	\$0
TECNOLOGIA (DESENVOLVIMENTO)	\$0	\$0
TECNOLOGIA (PRODUÇÃO)	\$0	\$30-80
EQUIPAMENTOS	\$0	\$0
CONTAS DESENVOLVEDOR	\$0	\$25-99/ANO
TOTAL	\$0	\$55-179/MÊS

Observação: O projeto atual utiliza apenas recursos gratuitos. Para produção, será necessário investir em serviços de backend e armazenamento em nuvem, além das taxas de desenvolvedor das lojas de aplicativos.

12. CONCLUSÃO

O projeto **GCD - Gerenciamento de Componentes Defeituosos** foi desenvolvido com sucesso, atendendo aos requisitos estabelecidos e proporcionando uma solução completa para o gerenciamento de ferramentas e defeitos em laboratórios educacionais.

A aplicação mobile desenvolvida em **React Native** com **Expo** oferece:

- **Interface intuitiva** e moderna
- Sistema robusto de **gerenciamento** de usuários
- **Funcionalidades completas de CRUD** para ferramentas e salas
- Sistema eficiente de **relatório e validação** de defeitos

- **Dashboard** analítico com visualizações gráficas
- **Armazenamento local** persistente
- **Design system consistente** e profissional

O sistema está pronto para uso em **ambiente** de desenvolvimento e pode ser facilmente adaptado para produção com a implementação de um backend real e serviços em nuvem.

12.1 Próximos Passos

Para evoluir o projeto para produção, recomenda-se:

1. Implementação de backend real (Node.js, Python, etc.)
2. Integração com banco de dados (PostgreSQL, MongoDB)
3. Implementação de autenticação JWT
4. Sincronização de dados em nuvem
5. Sistema de notificações push
6. Testes automatizados completos
7. Publicação nas lojas de aplicativos

Documento elaborado em: 15/11/2025

Versão: 1.0
